

Hello World RESTful Web Service

Write a REST service in the spring learn application created earlier, that returns the text "Hello World!!" using Spring Web Framework. Refer details below:

Method: GET

URL: /hello

Controller: com.cognizant.spring-learn.controller.HelloController

Method Signature: public String sayHello()

Method Implementation: return hard coded string "Hello World!!"

Sample Request: <http://localhost:8083/hello> Sample Response: Hello World!!

IMPORTANT NOTE: Don't forget to include start and end log in the sayHello() method.

Try the URL <http://localhost:8083/hello> in both chrome browser and postman.

SME to explain the following aspects:

- In network tab of developer tools show the HTTP header details received
- In postman click on "Headers" tab to view the HTTP header details received

Aim

To develop a RESTful web service using Spring Boot that returns the text "Hello World!!" when accessed through a GET request.

Procedure

1. Copy the previous Spring Boot project.
2. Configure the server port as 8083 in application.properties.
3. Create a package named controller.
4. Create HelloController.java.
5. Annotate the class with `@RestController`.
6. Create a GET endpoint using `@GetMapping("/hello")`.
7. Implement the sayHello() method to return "Hello World!!".
8. Add START and END log messages inside the method.
9. Run the application.

10. Test the endpoint in a browser and Postman.

Code

```
package spring_learn.controller;

import org.slf4j.Logger; import org.slf4j.LoggerFactory; import
org.springframework.web.bind.annotation.GetMapping; import
org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")
    public String sayHello() {

        LOGGER.info("START");

        LOGGER.info("END");

        return "Hello World!!";
    }
}
```

Output URL

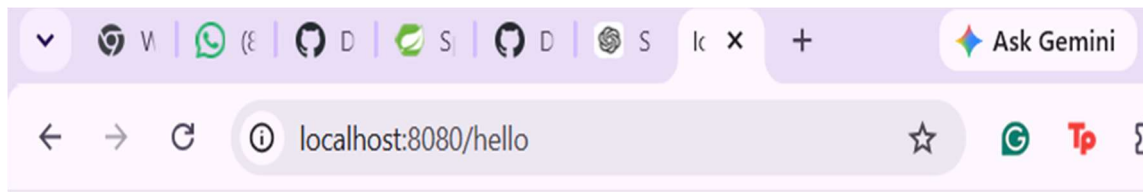
http://localhost:8083/hello

Response

Hello World!!

Result

A RESTful web service was successfully created using Spring Boot. The /hello endpoint returned "Hello World!!", and logging verified the execution of the sayHello() method.



Hello World!!

REST - Country Web Service

Write a REST service that returns India country details in the earlier created spring learn application.

URL: /country

Controller: com.cognizant.spring-learn.controller.CountryController

Method Annotation: @RequestMapping

Method Name: getCountryIndia()

Method Implementation: Load India bean from spring xml configuration and return

Sample Request: http://localhost:8083/country Sample Response:

```
{  
  "code": "IN",  
  "name": "India"  
}
```

SME to explain the following aspects:

- What happens in the controller method?
- How the bean is converted into JSON response?
- In network tab of developer tools show the HTTP header details received
- In postman click on "Headers" tab to view the HTTP header details received

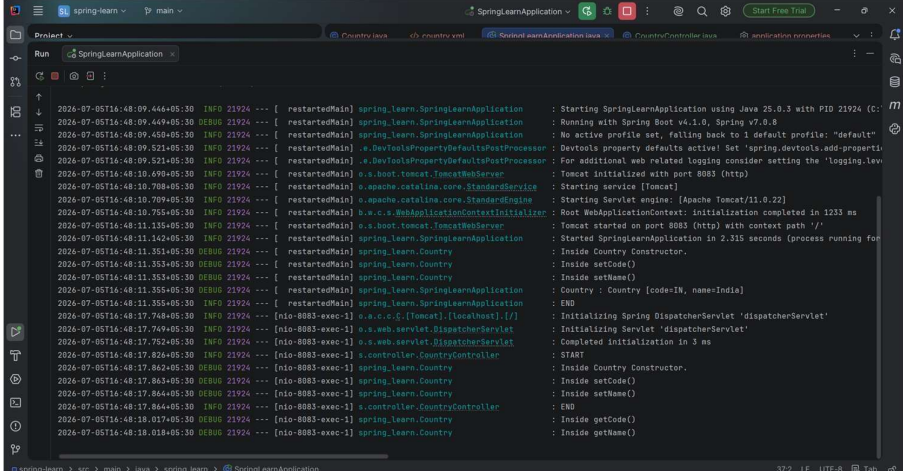
Aim

To develop a RESTful web service that returns the details of India by loading the Country bean from the Spring XML configuration file.

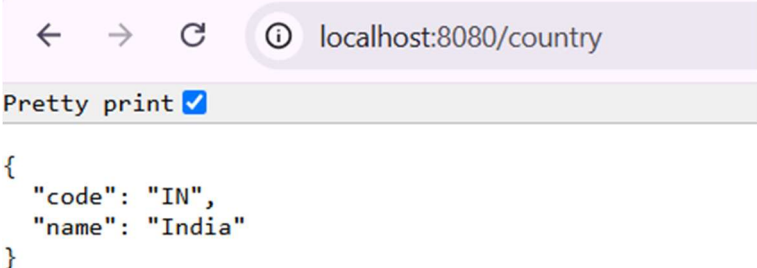
Procedure

1. Copy the previous Spring Boot project.
2. Copy Country.java and country.xml from the Spring Core exercise.
3. Create CountryController.java.
4. Annotate the class with `@RestController`.
5. Create a method `getCountryIndia()` using `@RequestMapping("/country")`.
6. Load the Country bean using `ClassPathXmlApplicationContext`.
7. Return the Country object.
8. Run the application.
9. Test the endpoint using a browser and Postman.

OUTPUT:



```
2024-07-05T16:48:09.444+05:30 INFO 21924 --- [ restartedMain ] spring.learn.SpringLearnApplication : Starting SpringLearnApplication using Java 25.0.3 with PID 21924 (C:\
2024-07-05T16:48:09.449+05:30 DEBUG 21924 --- [ restartedMain ] spring.learn.SpringLearnApplication : Running with Spring Boot v4.1.0, Spring v7.0.8
2024-07-05T16:48:09.450+05:30 INFO 21924 --- [ restartedMain ] spring.learn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2024-07-05T16:48:09.521+05:30 INFO 21924 --- [ restartedMain ] o.s.DevToolsPropertyDefaultsPostProcessor : DevTools property defaults active! Set 'spring.devtools.add-properties'
2024-07-05T16:48:09.521+05:30 INFO 21924 --- [ restartedMain ] o.s.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level
2024-07-05T16:48:10.490+05:30 INFO 21924 --- [ restartedMain ] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8083 (http)
2024-07-05T16:48:10.708+05:30 INFO 21924 --- [ restartedMain ] o.s.apache.catalina.core.StandardService : Starting service [Tomcat]
2024-07-05T16:48:10.709+05:30 INFO 21924 --- [ restartedMain ] o.s.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2024-07-05T16:48:11.353+05:30 INFO 21924 --- [ restartedMain ] o.s.w.s.web.application.ContextInitializer : Root WebApplicationContext: initialization completed in 1233 ms
2024-07-05T16:48:11.354+05:30 INFO 21924 --- [ restartedMain ] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8083 (http) with context path '/'
2024-07-05T16:48:11.354+05:30 INFO 21924 --- [ restartedMain ] spring.learn.SpringLearnApplication : Started SpringLearnApplication in 2.315 seconds (process running for
2024-07-05T16:48:11.351+05:30 DEBUG 21924 --- [ restartedMain ] spring.learn.Country : Inside Country Constructor.
2024-07-05T16:48:11.353+05:30 DEBUG 21924 --- [ restartedMain ] spring.learn.Country : Inside setCode()
2024-07-05T16:48:11.353+05:30 DEBUG 21924 --- [ restartedMain ] spring.learn.Country : Inside setName()
2024-07-05T16:48:11.354+05:30 DEBUG 21924 --- [ restartedMain ] spring.learn.SpringLearnApplication : Country : Country [code=IN, name=India]
2024-07-05T16:48:11.355+05:30 INFO 21924 --- [ restartedMain ] spring.learn.SpringLearnApplication : END
2024-07-05T16:48:17.748+05:30 INFO 21924 --- [nio-8083-exec-1] o.a.c.c.([Tomcat].[localhost].[/]) : Initializing Spring DispatcherServlet 'dispatcherServlet'
2024-07-05T16:48:17.749+05:30 INFO 21924 --- [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2024-07-05T16:48:17.752+05:30 INFO 21924 --- [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 3 ms
2024-07-05T16:48:17.824+05:30 INFO 21924 --- [nio-8083-exec-1] s.controller.CountryController : START
2024-07-05T16:48:17.824+05:30 DEBUG 21924 --- [nio-8083-exec-1] spring.learn.Country : Inside Country Constructor.
2024-07-05T16:48:17.863+05:30 DEBUG 21924 --- [nio-8083-exec-1] spring.learn.Country : Inside setCode()
2024-07-05T16:48:17.864+05:30 DEBUG 21924 --- [nio-8083-exec-1] spring.learn.Country : Inside setName()
2024-07-05T16:48:17.864+05:30 INFO 21924 --- [nio-8083-exec-1] s.controller.CountryController : END
2024-07-05T16:48:18.074+05:30 DEBUG 21924 --- [nio-8083-exec-1] spring.learn.Country : Inside getCode()
2024-07-05T16:48:18.074+05:30 DEBUG 21924 --- [nio-8083-exec-1] spring.learn.Country : Inside getName()
```



```
{
  "code": "IN",
  "name": "India"
}
```

REST - Get country based on country code

Write a REST service that returns a specific country based on country code. The country code should be case insensitive.

Controller: `com.cognizant.spring-learn.controller.CountryController`

Method Annotation: `@GetMapping("/countries/{code}")`

Method Name: `getCountry(String code)`

Method Implementation: `Invoke countryService.getCountry(code)`

Service Method: `com.cognizant.spring-learn.service.CountryService.getCountry(String code)`

Service Method Implementation:

- Get the country code using `@PathVariable`
- Get country list from `country.xml`
- Iterate through the country list
- Make a case insensitive matching of country code and return the country.
- Lambda expression can also be used instead of iterating the country list

Sample Request: `http://localhost:8083/country/in`

Sample Response:

```
{  
  "code": "IN",  
  "name": "India"  
}
```

Aim

To develop a RESTful web service that returns the details of a specific country based on the country code provided in the URL using Spring Boot.

Procedure

1. Copy the previous Spring Boot project.
2. Update `country.xml` to store a list of countries (India, United States, Germany, and Japan).
3. Create a `CountryService` class.

4. Load the country list from the Spring XML configuration file.
5. Search for the country using the country code provided in the URL.
6. Perform a case-insensitive comparison using `equalsIgnoreCase()`.
7. Create a REST endpoint using `@GetMapping("/country/{code}")` (or `@GetMapping("/countries/{code}")` if you followed the handout exactly).
8. Return the matching `Country` object.
9. Run the application.
10. Test the endpoint using a browser and Postman.

OUTPUT:

Pretty print ☐

```
{"code": "IN", "name": "India"}
```

← → ↻ ⓘ localhost:8080/countries

Pretty print ☒

```
[
  {
    "code": "IN",
    "name": "India"
  },
  {
    "code": "US",
    "name": "United States"
  },
  {
    "code": "DE",
    "name": "Germany"
  },
  {
    "code": "JP",
    "name": "Japan"
  }
]
```

Mon Jul 13 18:57:28 IST 2026

There was an unexpected error (type=Not Found, status=404).

Country not found

